



Formação Backend: Python & FastAPI — O Caminho Sannin de Orochimaru

Projeto Mushi Bingo

Aula Anterior

- Definição de back-end
- Definição de API
- Definição do protocolo HTTP
 - Verbos HTTP
 - Status Code
- Definição de Endpoints
- Ambiente de Desenvolver
- CRUD



Boas práticas em Python

Estilo e legibilidade

- Indentação com 4 espaços
- Linha deve ter no máximo 79 caracteres
- Declarações
 - variáveis e funções em snake_case
 - classes em PascalCase
 - constantes em UPPER_SNAKE

Dica: Utilize ferramentas com `ruff` para garantir as boas práticas em Python

Banco de Dados (Revisão) - SGBD

É um conjunto organizado de dados relacionados, armazenado e gerenciado de forma que permita inserção, consulta, atualização e remoção eficientes.

Modelagem de Dados:

- conceitual
- lógico
- físico

Normalização:

- 1FN
- 2FN
- 3FN

Modelagem de Dados

Conceitual

Representação de alto nível dos dados e suas relações, independente de tecnologia.

- Objetivo: capturar entidades, atributos e relacionamentos entendidos pelos stakeholders.
- Independente do SGBD.

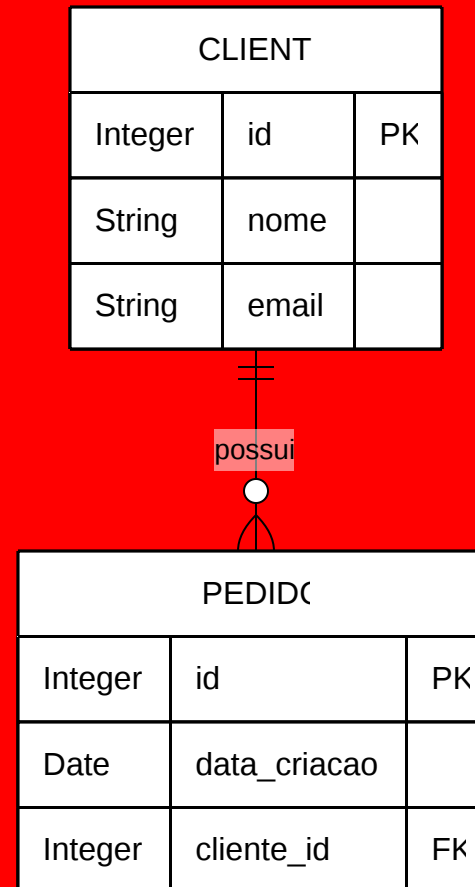
```
cliente:  
pedido
```

```
pedido:  
produto
```

Lógico

Mapeamento do modelo conceitual para estruturas lógicas (tabelas, colunas, chaves).

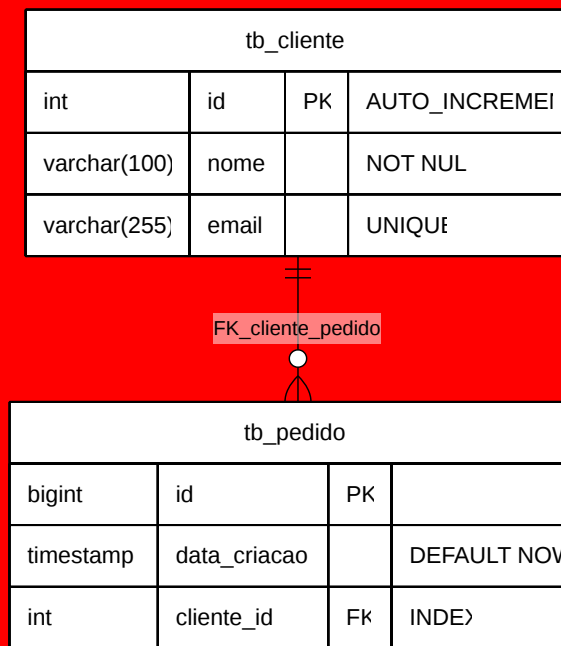
- Objetivo: definir chaves primárias/estrangeiras, cardinalidades e tipos de dados abstratos.
- Independente do SGBD.



Físico

Implementação concreta no SGBD escolhido, com decisões de armazenamento, índices e tipos concretos.

- Objetivo: otimizar desempenho, armazenamento e integridade no ambiente de produção
- Dependente do SGBD.



Normalização

Primeira Forma Normal (1FN)

Cada campo deve conter valores atômicos; eliminar grupos repetitivos/colunas multivaloradas.

- Objetivo: facilita consultas e evita redundância de dados dentro de uma tupla.

Não normalizado

CLIENT			
int	id	PK	
string	nome		
string	telefones		campo com múltiplos números

Normalizado

CLIENT		
int	id	PK
string	nome	

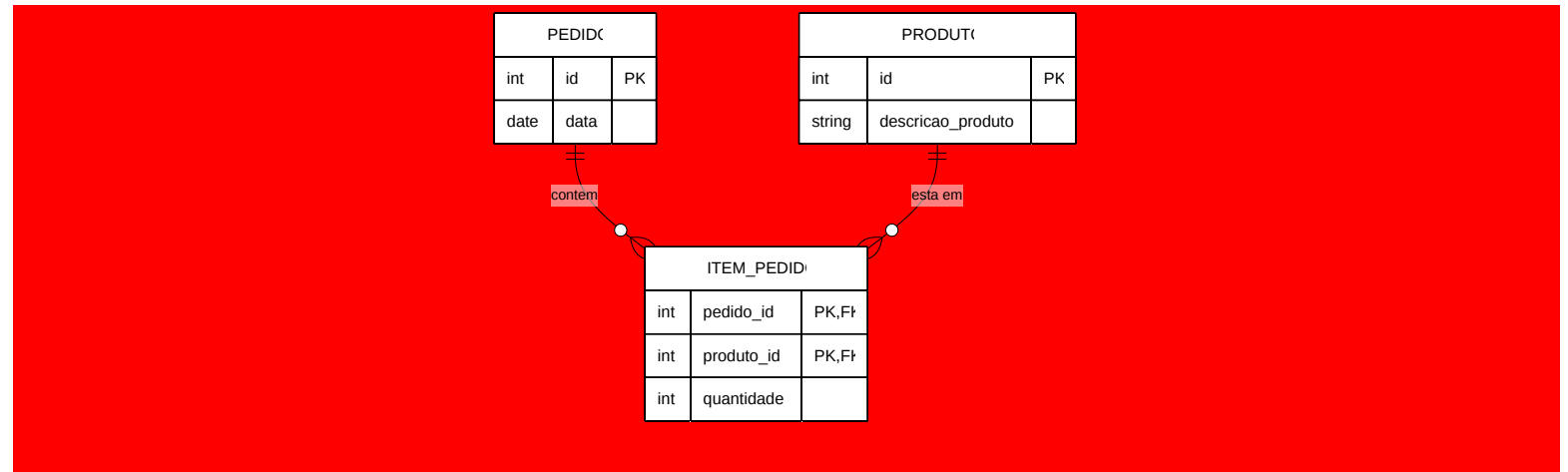
possui telefones

CLIENTE_TELEFO		
int	id	PK
int	cliente_id	FK
string	numero	

Segunda Forma Normal (2FN)

Estar em 1FN e todos os atributos não-chave dependerem funcionalmente da chave primária inteira (eliminar dependências parciais).

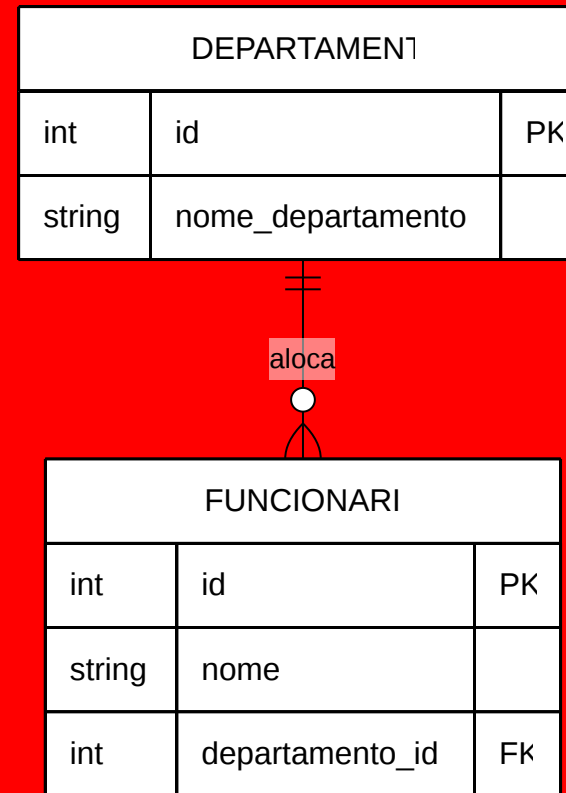
- Objetivo: evitar duplicação quando a chave primária é composta.



Terceira Forma Normal (3FN)

Estar em 2FN e nenhum atributo não-chave depender de outro atributo não-chave (eliminar dependências transitivas).

- Objetivo: reduzir anomalias de atualização e mantém integridade sem redundância desnecessária.



Comandos básicos de SQL

- criar

```
CREATE TABLE clientes (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE  
);
```

Comandos básicos de SQL

- listar

```
SELECT id, nome, email  
FROM clientes  
WHERE nome = 'João Silva';
```

Comandos básicos de SQL

- delatar

```
DELETE FROM clientes  
WHERE id = 1;
```

Comandos básicos de SQL

- inserir

```
INSERT INTO clientes (nome, email)  
VALUES ('João Silva', 'joao@email.com');
```

Comandos básicos de SQL

- atualizar

```
UPDATE clientes  
SET email = 'novoemail@email.com'  
WHERE id = 1;
```


Boas práticas para banco de dados

Nomenclatura: Regras Gerais

- Padrão Snake Case:
 - *Correto:* `data_nascimento`
 - *Incorreto:* `DataNascimento`, `dataNascimento`
- Idioma Unificado
- Caracteres Permitidos
- Palavras Reservadas

Nomenclatura: Tabelas

- Sempre no Singular:
 - *Correto:* cliente, item_pedido
- Evite prefixos como tb_ ou tbl_.
 - *Correto:* usuario
 - *Incorreto:* tb_usuario

Nomenclatura: Atributos (Colunas)

- Sempre no Singular:
 - *Correto:* nome, preco_unitario
- Evite Redundância
 - *Correto:* nome, cpf
 - *Incorreto:* nome_cliente, cpf_cliente
- Seja Descritivo e Claro
 - *Correto:* quantidade_estoque
 - *Incorreto:* qtd_est

Nomenclatura: Chaves (PK e FK)

- Chave Primária (PK): Utilizar simplesmente `id`.
- Chave Estrangeira (FK): Deve seguir a regra estrita de usar o nome da tabela referenciada no singular, seguido do sufixo `_id`.
 - *Exemplo em uma tabela de pedidos:* `cliente_id`, `produto_id`.

O projeto Mushi Bingo

Objetivo: Criar uma versão simplificada do MyAnimeList, permitindo cadastrar animes e acompanhar o status de cada um (assistindo, parado, assistido, planejo assistir).

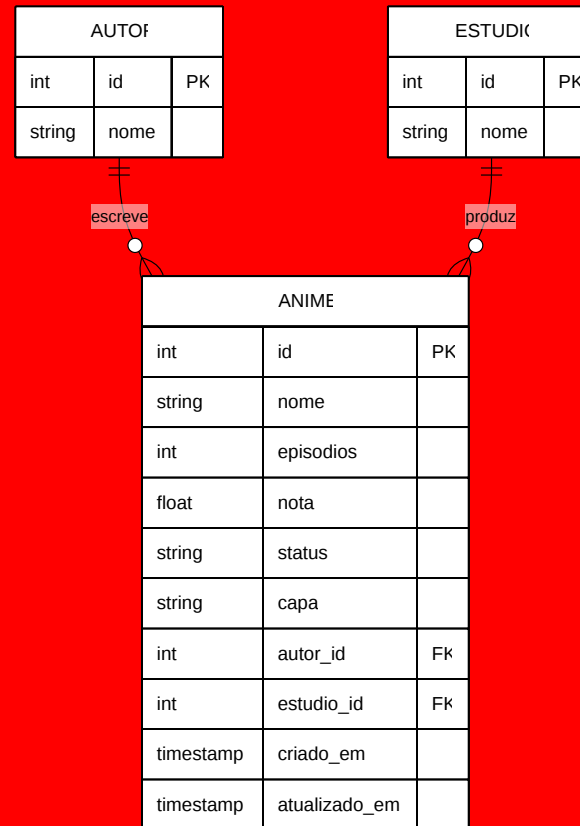


Diagrama RE

É uma representação visual que descreve a estrutura lógica de um banco de dados, facilitando a compreensão das regras de negócio pela equipe.

Componentes fundamentais:

- Entidades (tabelas)
- Atributos (colunas)
- Relacionamentos



Regras de negócio: Fluxo de Status

Fluxo de progresso:

- `planejo_assistir` -> `assistindo` -> `assistido`
 - *Regra:* Um anime só pode ser marcado como `assistido` depois de já ter passado por `assistindo`.

Fluxo de pausa:

- `assistindo` -> `parado` -> `assistindo`
 - *Regra:* Um anime `parado` pode voltar a `assistindo` a qualquer momento.

Regras de negócio: Nota

- A nota é opcional e vai de 0 a 10 (aceita casas decimais, ex: 8.5).
- Só faz sentido atribuir nota quando o status for assistindo ou assistido.
 - *Regra:* Se o status for planejo_assistir, a nota deve ser nula.

Regras de negócio: Cadastro

- O campo `nome` do anime é obrigatório.
- `autor_id` e `estudio_id` são opcionais (nem todo anime tem essa informação cadastrada), mas quando informados devem referenciar um registro existente em `autor` / `estudio`.
- O campo `nome` em `autor` e `estudio` deve ser único, evitando cadastros duplicados do mesmo autor ou estúdio.
- O campo `episodios` representa o total de episódios da obra (não o progresso assistido).
- `criado_em` é preenchido automaticamente na criação do registro (`DEFAULT NOW()`).
- `atualizado_em` é atualizado automaticamente a cada modificação do registro.
- O campo `capa` é opcional e armazena apenas o **caminho/URL da imagem** salva no servidor. O arquivo em si não fica dentro do banco de dados.

Endpoints (Sugestão): Autores

- `GET /autores` : Lista autores.
- `GET /autores/{id}` : Retorna um autor com os animes vinculados.
- `POST /autores` : Cria um novo autor.
- `PATCH /autores/{id}` : Atualiza dados do autor.

Endpoints (Sugestão): Estúdios

- `GET /estudios` : Lista estúdios.
- `GET /estudios/{id}` : Retorna um estúdio com os animes vinculados.
- `POST /estudios` : Cria um novo estúdio.
- `PATCH /estudios/{id}` : Atualiza dados do estúdio.

Endpoints (Sugestão): Animes

- `GET /animes` : Lista animes (Filtros: `?status=`, `?autor_id=`, `?estudio_id=`).
- `GET /animes/{id}` : Retorna um anime específico.
- `POST /animes` : Cria um novo anime.
- `PATCH /animes/{id}` : Atualiza dados (incluindo status e nota).
- `DELETE /animes/{id}` : Remove um anime da lista.
- `POST /animes/{id}/capa` : Recebe a imagem de capa do anime e salva o caminho no campo `capa`.
 - *Nota:* implementado com `UploadFile` do FastAPI.

Estrutura de Pasta (Projeto)

```
myanimelist
├── app
│   ├── main.py
│   ├── models
│   ├── routers
│   ├── schema
│   └── settings.py
├── test
├── migrations
├── database.db
├── alembic.ini
└── .env
```

ORM (Object-Relational Mapping)

É uma técnica de programação (ferramenta) que permite interagir diretamente com um banco de dados usando o paradigma de Orientação a Objetos da sua linguagem de programação, em vez de escrever consultas SQL puras.

- Abstração de banco de dados
- Segurança
- Eficiência no desenvolvimento

SQLAlchemy

É o ORM mais robusto e tradicional do ecossistema Python. Ele abstrai a complexidade do banco de dados relacional, permitindo que você manipule tabelas e registros como se fossem classes e objetos Python nativos.

- Mapeamento Objeto-Relacional
- Abstração de Dialeto
- Gerenciamento de Sessão

Para instalar

```
# No linux e Windows  
uv add sqlalchemy
```


Migrations

É o conceito de controle de versão aplicado ao esquema (estrutura) do banco de dados. As migrações registram cada evolução da estrutura em arquivos de código ordenados cronologicamente.

- Histórico de Evolução
- Consistência de Ambientes
- Preservação de Dados

Alembic

É a ferramenta oficial de migração de banco de dados projetada especificamente para funcionar em conjunto com o SQLAlchemy. Ele automatiza o processo de criação e aplicação das migrações.

- Autogeração de scripts
- Controle de direção (Upgrade/Downgrade)
- Automação de deploy

Para instalar

```
# No linux e Windows
uv add alembic

uv run alembic init migrations

uv run alembic revision --autogenerate -m "mensagem"
```



Atividade

- Leia as regras de negócios e tente implementar os recursos (endpoints)
- Tente criar por si mesmo as tabelas do banco de dados
- Tente refazer o diagrama do banco de dados conforme seu gosto (assim como as regras de negócio e endpoints)

Próximos tópicos

- Conectar Banco de Dados
- Router
- Criar Recursos
- RedirectResponse
- Injeção de Dependência
- Middleware

Referencias

- FastAPI do Zero
- FastAPI
- UV
- Pydantic
- Métodos de requisição HTTP
- REST: Princípios e boas práticas
- 5 dicas para fazer APIs melhores
- Alembic
- SQLAlchemy
- Ambiente Python Moderno 2025: UV, Ruff, Pyright, pyproject.toml e VS Code
- Curso de Modelagem de Dados



Até a próxima!